1. creación de la base de datos y las tablas

None

```
CREATE TABLE Jugadores (
    jugador_id      INT AUTO_INCREMENT PRIMARY KEY,
    nombre_usuario  VARCHAR(50)  NOT NULL UNIQUE,
    email           VARCHAR(100) NOT NULL UNIQUE,
    nivel           INT NOT NULL DEFAULT 1,
    experiencia     INT NOT NULL DEFAULT 0,
    monedas         INT NOT NULL DEFAULT 0,
    region          VARCHAR(20) NOT NULL DEFAULT 'AMERICA',
    fecha_registro  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE Juegos (
    juego_id        INT AUTO_INCREMENT PRIMARY KEY,
    nombre          VARCHAR(80) NOT NULL,
    genero          VARCHAR(40),
    fecha_lanzamiento DATE
) ENGINE=InnoDB;

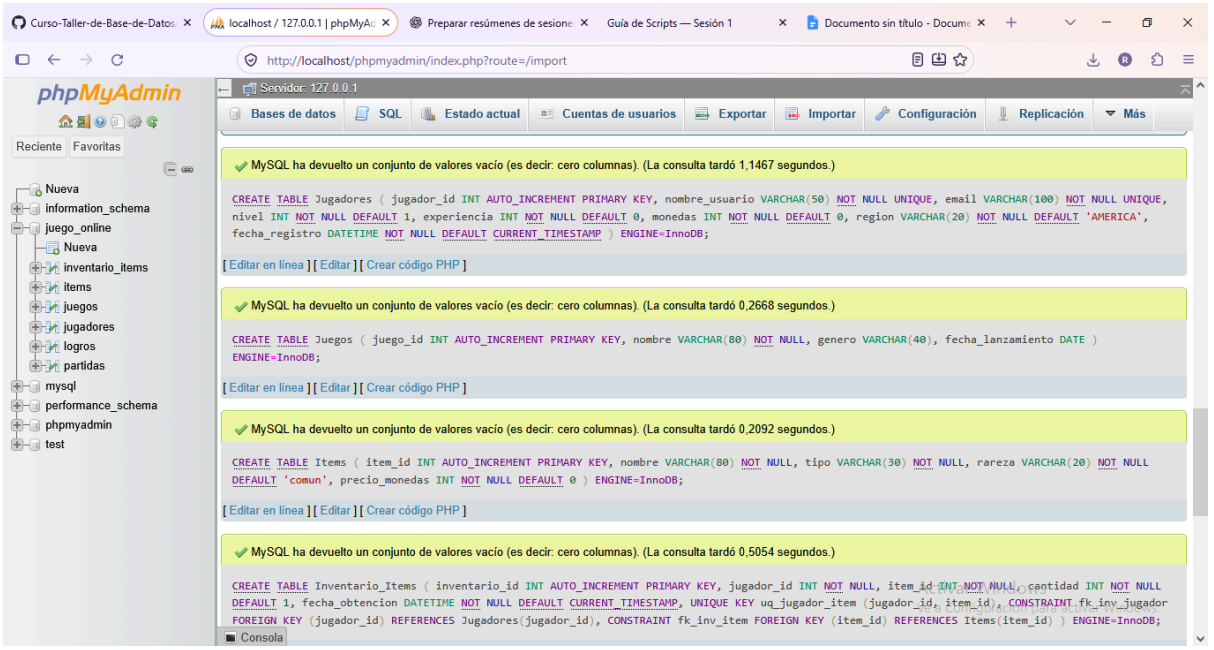
CREATE TABLE Items (
    item_id         INT AUTO_INCREMENT PRIMARY KEY,
    nombre          VARCHAR(80) NOT NULL,
    tipo            VARCHAR(30) NOT NULL,
    rareza          VARCHAR(20) NOT NULL DEFAULT 'comun',
    precio_monedas  INT NOT NULL DEFAULT 0
) ENGINE=InnoDB;

CREATE TABLE Inventario_Items (
    inventario_id   INT AUTO_INCREMENT PRIMARY KEY,
    jugador_id      INT NOT NULL,
    item_id         INT NOT NULL,
    cantidad        INT NOT NULL DEFAULT 1,
    fecha_obtencion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_jugador_item (jugador_id, item_id),
    CONSTRAINT fk_inv_jugador FOREIGN KEY (jugador_id) REFERENCES Jugadores(jugador_id),
    CONSTRAINT fk_inv_item    FOREIGN KEY (item_id) REFERENCES Items(item_id)
) ENGINE=InnoDB;

CREATE TABLE Partidas (
    partida_id      INT AUTO_INCREMENT PRIMARY KEY,
    jugador_id      INT NOT NULL,
    juego_id        INT NOT NULL,
    puntaje         INT NOT NULL DEFAULT 0,
    duracion_seg    INT NOT NULL DEFAULT 0,
    resultado       VARCHAR(20) NOT NULL DEFAULT 'jugada',
    fecha           DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_partida_jugador FOREIGN KEY (jugador_id) REFERENCES Jugadores(jugador_id),
    CONSTRAINT fk_partida_juego   FOREIGN KEY (juego_id) REFERENCES Juegos(juego_id)
) ENGINE=InnoDB;

CREATE TABLE Logros (
    logro_id        INT AUTO_INCREMENT PRIMARY KEY,
    jugador_id      INT NOT NULL,
    nombre_logro    VARCHAR(80) NOT NULL,
    fecha_obtenido  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_logro_jugador FOREIGN KEY (jugador_id) REFERENCES Jugadores(jugador_id)
) ENGINE=InnoDB;

CREATE INDEX idx_partidas_jugador ON Partidas(jugador_id);
CREATE INDEX idx_inventario_jugador ON Inventario_Items(jugador_id);
```



aqui aprendi como crear la base de datos y las tablas con sus respectivos valores como varchar, null,primary key y algo interesante en esta parte seria la auto incrementacion de numero en cada tabla ejemplo:jugador_id INT AUTO_INCREMENT PRIMARY KEY donde al registrar jugadores, el sistema genera identificadores automáticamente.

2. insertar datos de prueba:

None

```
INSERT INTO Jugadores (nombre_usuario, email, nivel, experiencia, monedas, region) VALUES
('DragonSlayer99', 'dragon99@game.com', 12, 340, 1500, 'AMERICA'),
('ShadowNinja',    'shadow@game.com',    8,  120, 900,  'AMERICA'),
('PixelQueen',     'pixel@game.com',    20,  980, 5000, 'EUROPA'),
('NoobMaster',     'noob@game.com',      1,   10,  50,  'EUROPA'),
('RocketFox',      'fox@game.com',      15,  600, 2200, 'AMERICA');

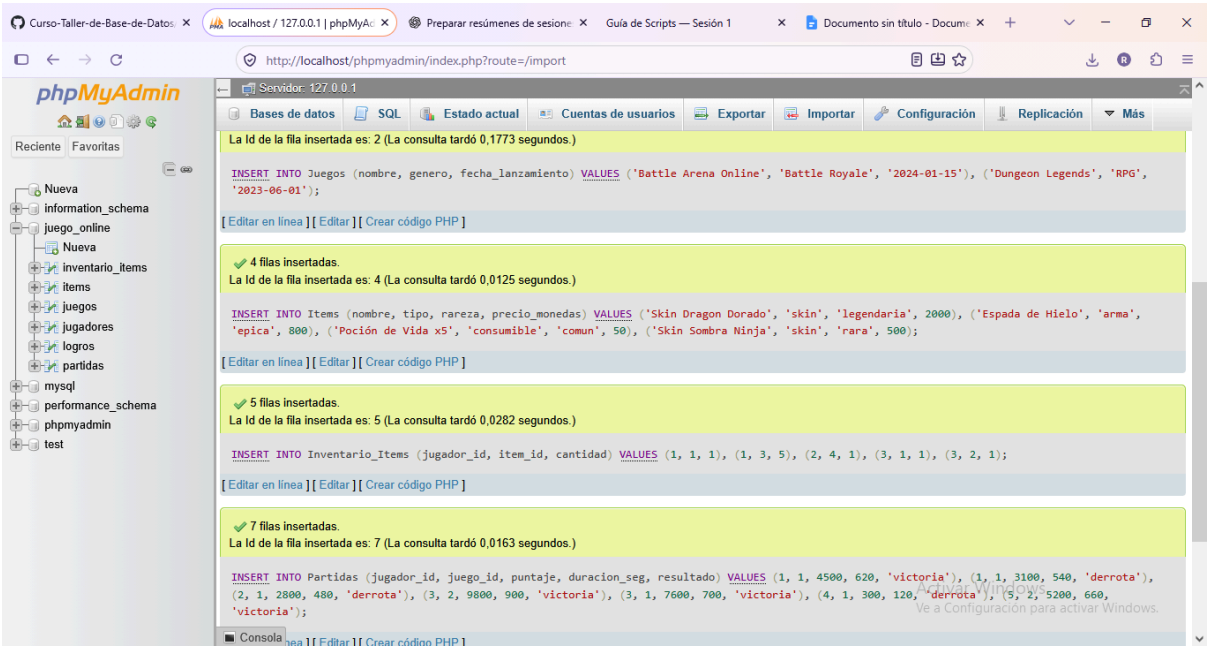
INSERT INTO Juegos (nombre, genero, fecha_lanzamiento) VALUES
('Battle Arena Online', 'Battle Royale', '2024-01-15'),
('Dungeon Legends',     'RPG',           '2023-06-01');

INSERT INTO Items (nombre, tipo, rareza, precio_monedas) VALUES
('Skin Dragon Dorado',  'skin',  'legendaria', 2000),
('Espada de Hielo',     'arma',  'epica',      800),
('Poción de Vida x5',   'consumible', 'comun', 50),
('Skin Sombra Ninja',   'skin',  'rara',       500);

INSERT INTO Inventario_Items (jugador_id, item_id, cantidad) VALUES
(1, 1, 1),
(1, 3, 5),
(2, 4, 1),
(3, 1, 1),
(3, 2, 1);

INSERT INTO Partidas (jugador_id, juego_id, puntaje, duracion_seg, resultado) VALUES
(1, 1, 4500, 620, 'victoria'),
(1, 1, 3100, 540, 'derrota'),
(2, 1, 2800, 480, 'derrota'),
(3, 2, 9800, 900, 'victoria'),
(3, 1, 7600, 700, 'victoria'),
(4, 1, 300,  120, 'derrota'),
(5, 2, 5200, 660, 'victoria');

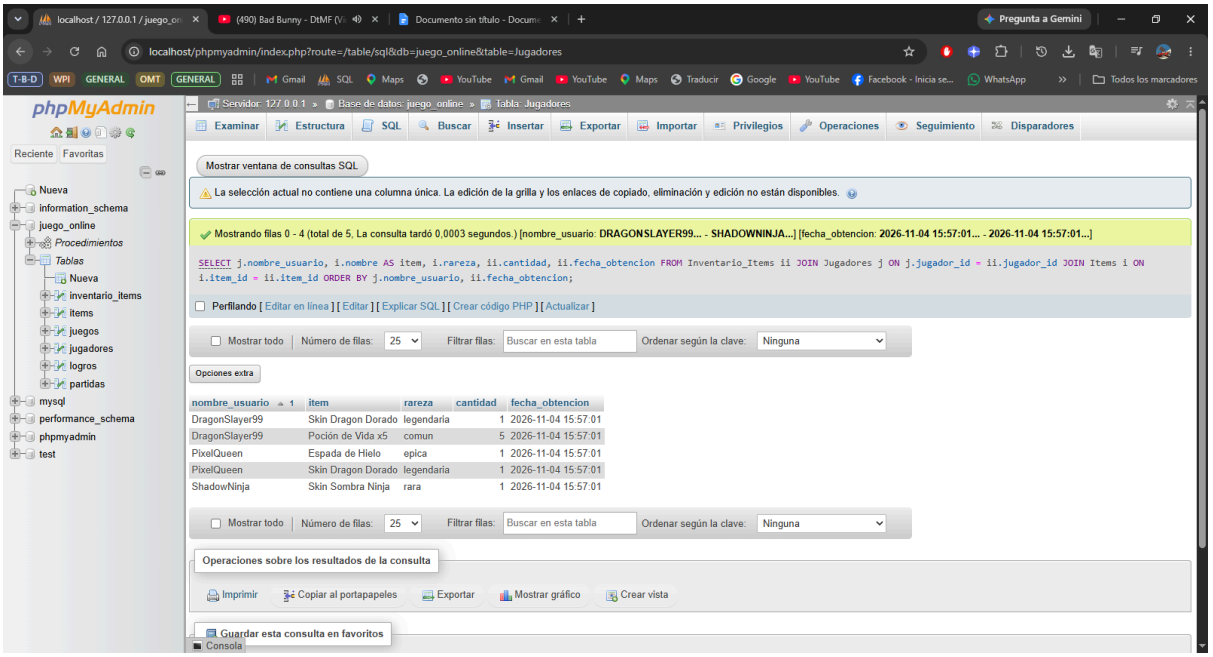
INSERT INTO Logros (jugador_id, nombre_logro) VALUES
(3, 'Campeón Invicto'),
(1, 'Coleccionista Legendario'),
(5, 'Explorador de Mazmorras');
```



aqui insertamos datos de prueba, datos para cada tabla, donde seleccionamos la tabla y inseetamos los datos tomando en cuenta el orden de cada tabla de que datos estan permitidos.

3. consultas leaderboard:

3.1:tabla leaderboard:

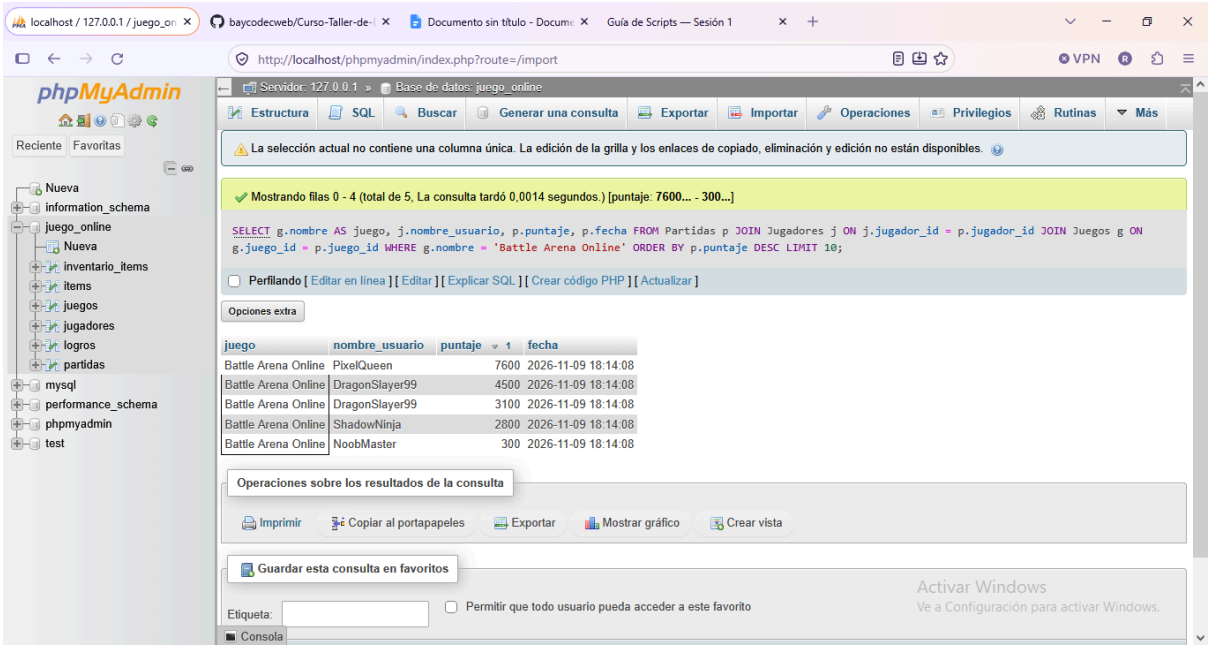


en esta consulta nos permite identificar que jugador consiguio el puntaje mas alto y conocer su nivel y region

3.2:Leaderboard por juego específico

None

```
SELECT
    g.nombre AS juego,
    j.nombre_usuario,
    p.puntaje,
    p.fecha
FROM Partidas p
JOIN Jugadores j ON j.jugador_id = p.jugador_id
JOIN Juegos g ON g.juego_id = p.juego_id
WHERE g.nombre = 'Battle Arena Online'
ORDER BY p.puntaje DESC
LIMIT 10;
```

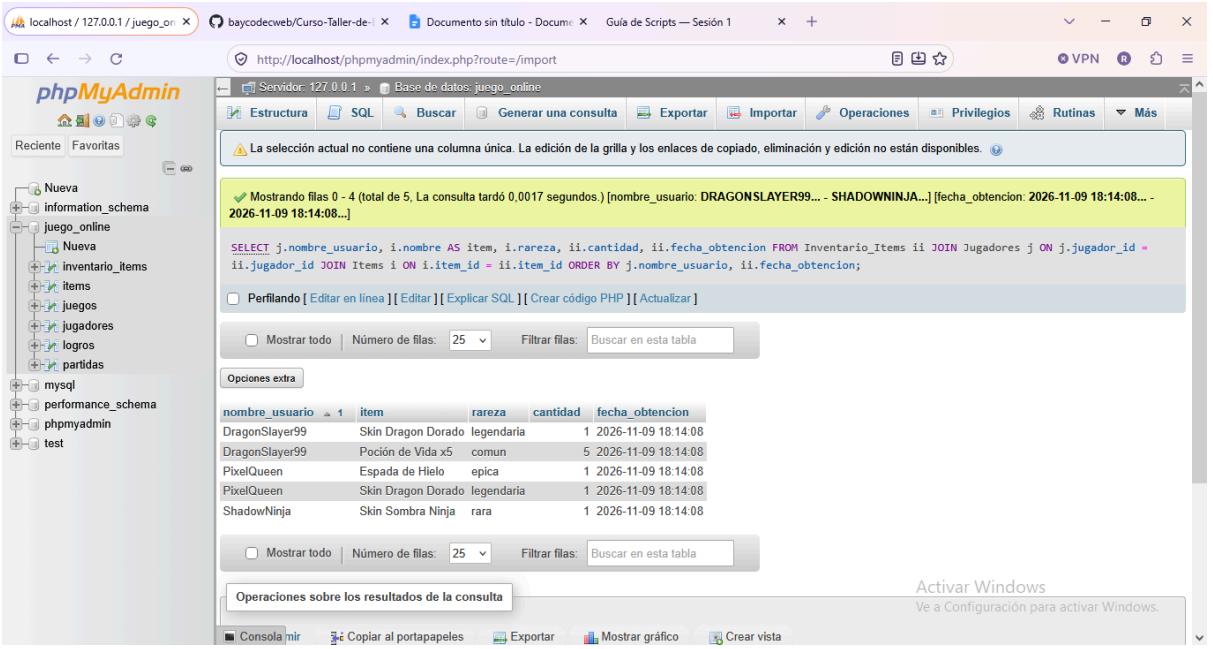


esto es lo mismo que la anterior consulta solo que escogimos un juego especifico

3.3:Catálogo de items comprado

None

```
SELECT
    j.nombre_usuario,
    i.nombre AS item,
    i.rareza,
    ii.cantidad,
    ii.fecha_obtencion
FROM Inventario_Items ii
JOIN Jugadores j ON j.jugador_id = ii.jugador_id
JOIN Items i ON i.item_id = ii.item_id
ORDER BY j.nombre_usuario, ii.fecha_obtencion;
```

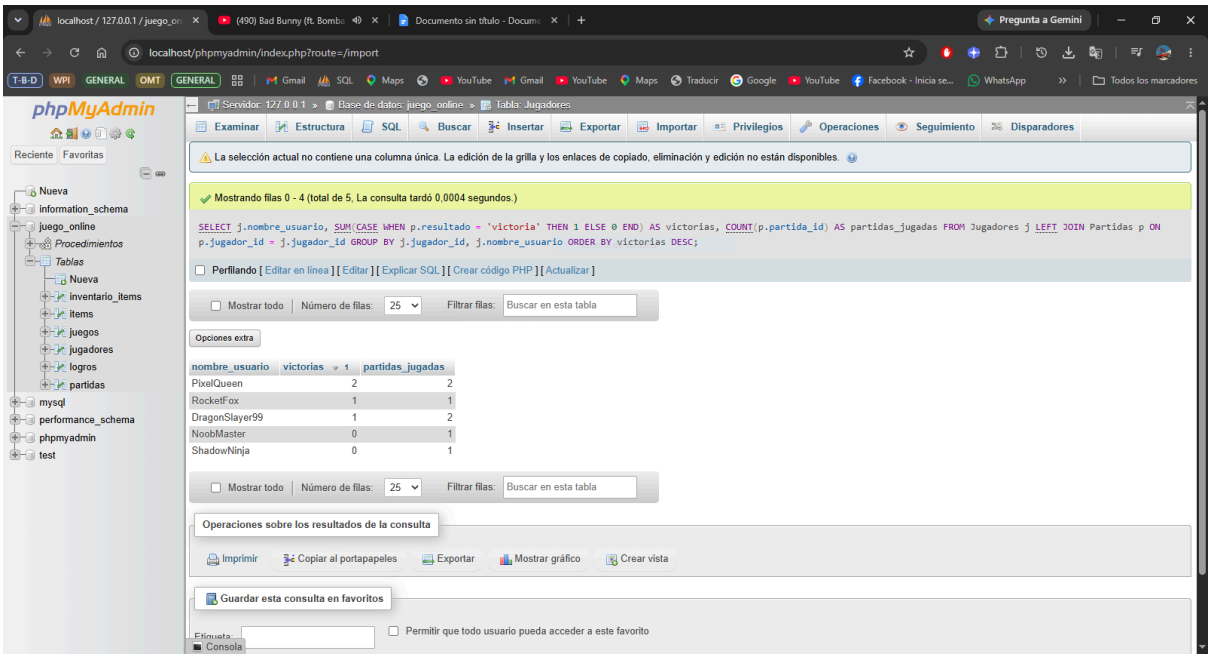


no es un ranking, pero usa el mismo patrón de join que verías en un historial de compras o un inventario visible para el jugador.

3.4:Ranking de victorias por jugador

None

```
SELECT
    j.nombre_usuario,
    SUM(CASE WHEN p.resultado = 'victoria' THEN 1 ELSE 0 END) AS victorias,
    COUNT(p.partida_id) AS partidas_jugadas
FROM Jugadores j
LEFT JOIN Partidas p ON p.jugador_id = j.jugador_id
GROUP BY j.jugador_id, j.nombre_usuario
ORDER BY victorias DESC;
```



en esta consulta nos da como resultado el rendimiento de cada jugador y quien tiene mas victorias
4:subir de nivel:

None

```
USE juego_online;

DROP PROCEDURE IF EXISTS subir_nivel_jugador;

DELIMITER //

CREATE PROCEDURE subir_nivel_jugador(
    IN p_jugador_id INT,
    IN p_experiencia_ganada INT
)
BEGIN
    DECLARE v_experiencia_total INT;
    DECLARE v_niveles_ganados INT;

    SELECT experiencia + p_experiencia_ganada INTO v_experiencia_total
    FROM Jugadores WHERE jugador_id = p_jugador_id;

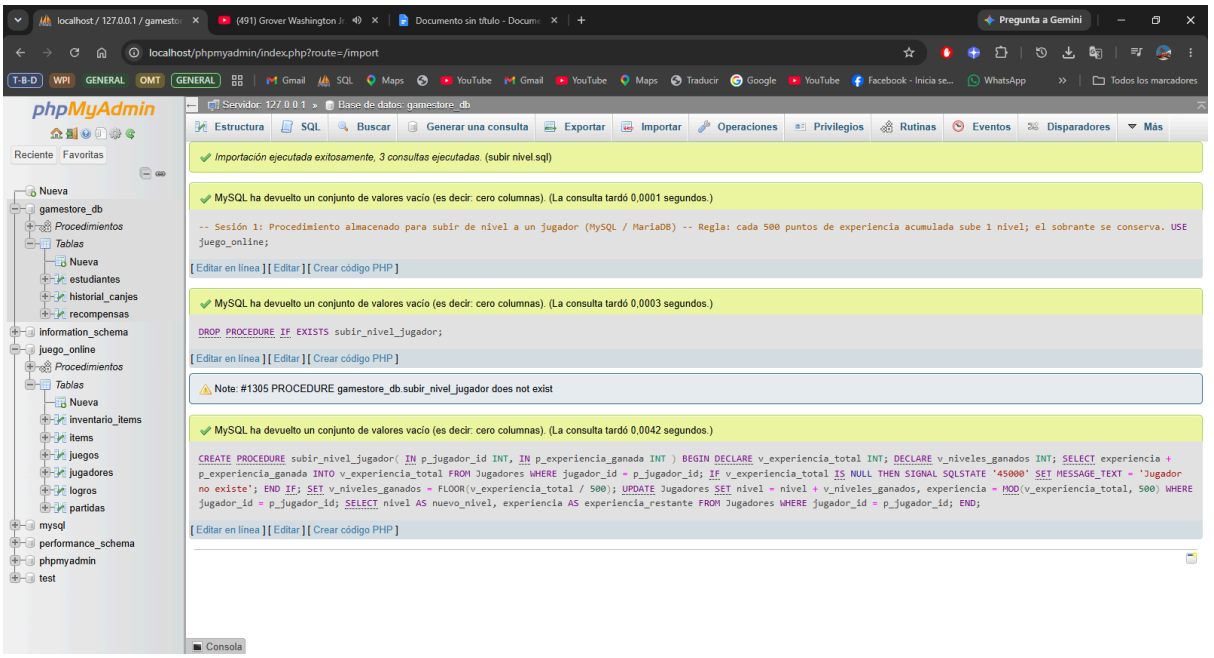
    IF v_experiencia_total IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Jugador no existe';
    END IF;

    SET v_niveles_ganados = FLOOR(v_experiencia_total / 500);

    UPDATE Jugadores
    SET nivel = nivel + v_niveles_ganados,
        experiencia = MOD(v_experiencia_total, 500)
    WHERE jugador_id = p_jugador_id;

    SELECT nivel AS nuevo_nivel, experiencia AS experiencia_restante
    FROM Jugadores WHERE jugador_id = p_jugador_id;
END //

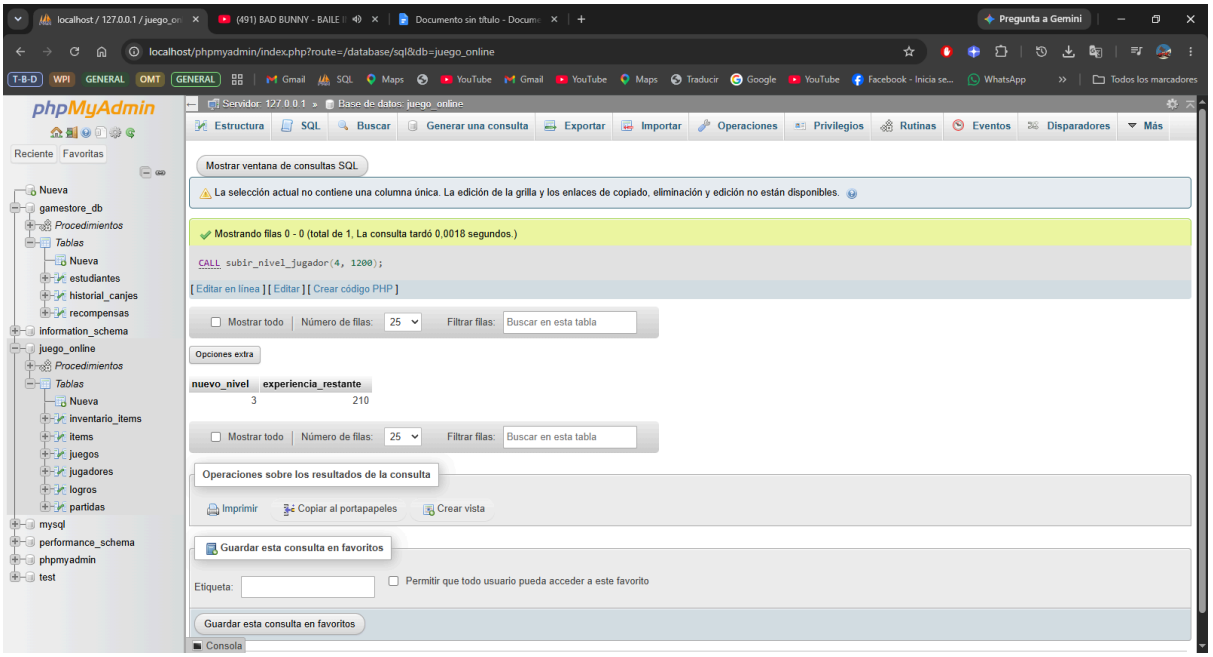
DELIMITER ;
```



en esta consulta nos permite subir automaticamente el nivel a un jugador cuando este acumula 500 puntos de experiencia y conservar los puntos.
4.1:

None

```
CALL subir_nivel_jugador(4, 1200);
```



con esta consulta nos permite visualizar a jugador numero 4 subir 2 niveles y conservar 200 puntos de experiencia